Twacha — Complete Project Blueprint

Skin & Dietary Wellness Tracker · Flutter + FastAPI · Final Version

---

Table of Contents

1. Project Summary & USP
2. Locked Tech Decisions
3. Full Architecture
4. API Contract (Locked)
5. Recommendation Engine — How It Works
6. CV Pipeline — What Gets Measured & How
7. 30-Day Adaptive Program
8. Privacy & Image Lifecycle
9. Pollution-Adjusted Skin Insights (USP)
10. Flutter App — Screen Map
11. Development Roadmap (4 Phases)
12. Backend — What Is Built & Working
13. Datasets & Data Sources
14. Evaluation Plan
15. Do / Don't

---

1. Project Summary & USP

Twacha (त्वacha — Sanskrit/Hindi for "skin") is a Flutter mobile app that tracks daily skin health using the smartphone camera, maps skin condition scores to personalized dietary recommendations, and adjusts advice based on local air quality. Designed and optimized for the Indian demographic (Fitzpatrick III–VI).

Core differentiator vs. competitors (LooksIQ, SkinPal, TroveSkin):

Competitor	How they work	Twacha's advantage
LooksIQ	Wraps a general-purpose vision LLM (OpenAI via OpenRouter) around a selfie	Domain-trained CV model on ACNE04; evidence-tagged, rule-based recommendations — not LLM free-form output
SkinPal / TroveSkin	Product recommendation funnels	No product selling; diet-first, holistic
All competitors	Ignore environmental context	Pollution-adjusted recommendations using real-time AQI — the Diet-Skin-Environment triangle

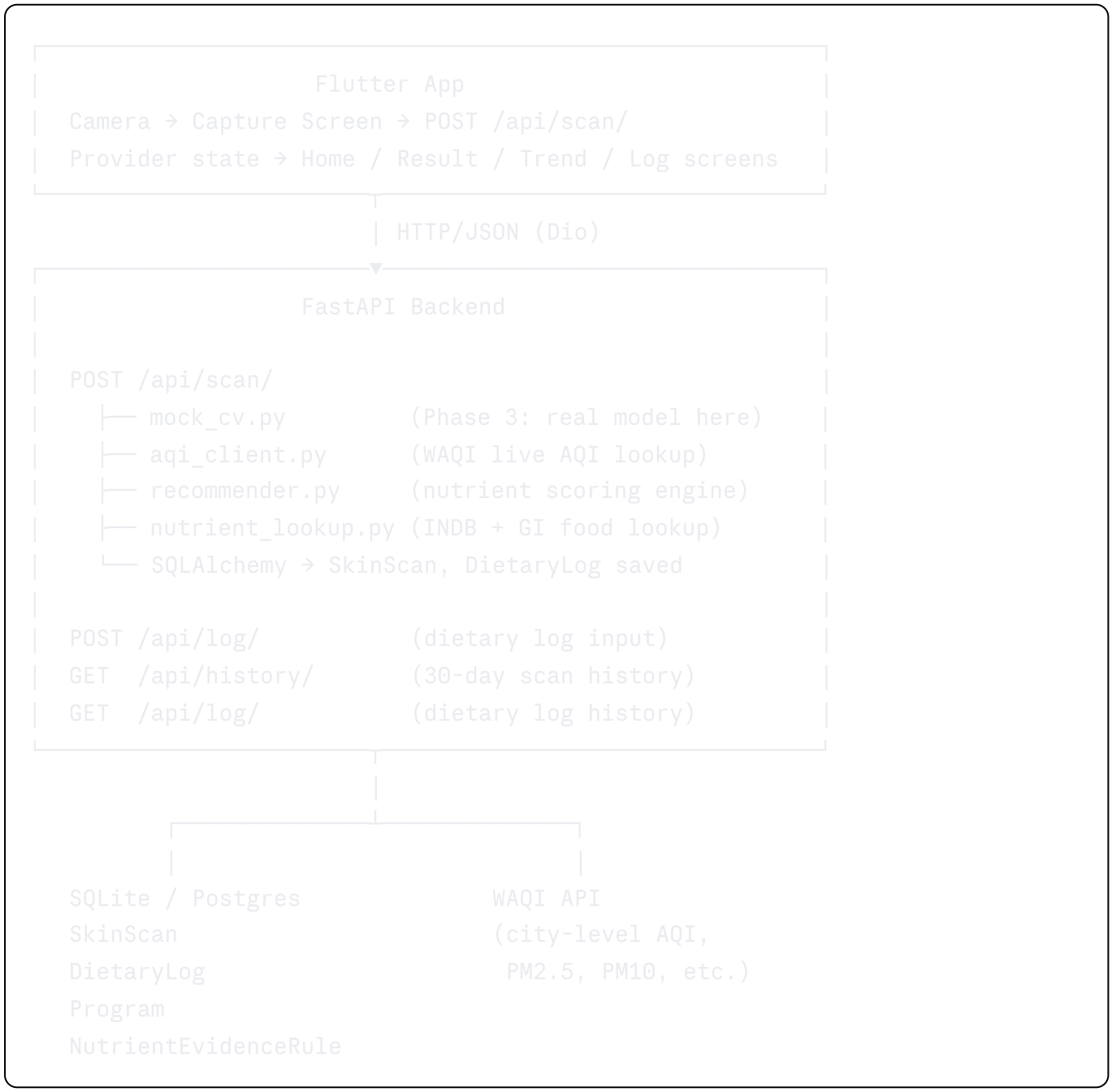
USP in one sentence: The only skin tracker that connects what you eat, what the air looks like today, and what your skin looked like yesterday — all tuned for Indian food and Indian skin.

2. Locked Tech Decisions

Layer	Choice	Reason
Mobile frontend	**Flutter**	Cross-platform (Android + iOS), single codebase
State management	**Provider**	Simpler than Riverpod/Bloc, right-sized for a 2-person team
Backend framework	**FastAPI**	Fast, async, auto-generates Swagger docs, Pydantic enforces the API contract automatically
ORM	**SQLAlchemy + Alembic**	Replaces Django's built-in ORM; Alembic handles migrations
Database (local dev)	**SQLite**	Zero setup for development
Database (production)	**Render Postgres** or **SQLite + Render Persistent Disk**	⚠️ Render free-tier has ephemeral filesystem — SQLite wipes on every redeploy. Decide this before Phase 4.
Hosting	**Render** (backend) + **Play Store internal testing** (Flutter)	Free-tier, git-push deploys
ML/DL framework	**PyTorch**	Matches ACNE04 reference implementations; YOLOv8 is PyTorch-native
AQI data	**WAQI (aqicn.org)**	Free token, 1000+ Indian cities, real-time PM2.5/PM10/NO2
HTTP client (Flutter)	**Dio**	Cleaner multipart upload handling than `http` package
Charts (Flutter)	**fl_chart**	7-day trend line chart
DB inspection tool	**DB Browser for SQLite**	Replaces Django admin for eyeballing data without writing UI

---

3. Full Architecture



What never touches the database: the raw image. Captured → processed in-memory → discarded. Only the derived score vector persists.

---

4. API Contract (Locked)

Both team members build against this shape. Field names, types, and enum values are fixed — do not drift from this in either codebase.

`POST /api/scan/`

Request: multipart form, one field `image` (jpeg/png). Optional for now (mock CV doesn't need it; real model will).

Response (201):

json

```
{
  "scan": {
    "id": 14,
    "created_at": "2026-09-14T10:32:00+05:30",
    "day_number": 7,
    "acne_score": 4.2,
    "redness_score": 3.1,
    "dryness_score": 2.0,
    "ita_value": 34.5
  },
  "environment": {
    "local_aqi": 187,
    "pm25": 92.4,
    "dominant_pollutant": "pm25",
    "aqi_band": "poor"
  },
  "nutrient_scores": {
    "low_glycemic": 6.627,
    "omega3": 5.590,
    "antioxidant_vit_e": 4.998,
    "zinc": 4.820,
    "healthy_fats": 4.325,
    "antioxidant_vit_c": 4.152,
    "dairy_reduce": 4.114,
    "vitamin_d": 4.055,
    "water_intake": 3.695,
    "probiotic": 3.395
  },
  "recommendations": [
    {
      "nutrient_key": "low_glycemic",
      "relevance_score": 6.627,
      "title": "Lower your glycemic load",
      "body": "High-GI foods spike insulin, increasing sebum production.",
      "nutrient_explanation": "...",
      "evidence_tier": "clinical",
      "source_note": "Fabbrocini et al. 2020 (AJCN)",
      "foods_to_add": [
        { "name": "Rajma (kidney beans)", "reason": "GI ~29", "gi": 29 },
        { "name": "Jowar roti", "reason": "GI ~55, swap for maida", "gi": 55 }
      ],
      "foods_to_reduce": [
        { "name": "White rice", "reason": "GI 72-80", "gi": 76 },
        { "name": "Poha", "reason": "Assumed healthy but GI ~70", "gi": 70 }
```

```
      ],
      "trend": "improving"
    }
  ]
}
```

`GET /api/history/?limit=30`

Response (200):

json

```
{
  "program": {
    "start_date": "2026-09-08",
    "current_day": 7,
    "total_days": 30
  },
  "scans": [
    {
      "id": 14,
      "created_at": "2026-09-14T10:32:00+05:30",
      "day_number": 7,
      "acne_score": 4.2,
      "redness_score": 3.1,
      "dryness_score": 2.0,
      "ita_value": 34.5,
      "local_aqi": 187
    }
  ]
}
```

`POST /api/log/`

Request (JSON):

json

```
{ "foods_eaten": ["white rice", "dal", "curd"], "log_date": "2026-09-14" }
```

Response (201): same shape back with `id` and `log_date`.

`GET /api/log/?limit=7`

Returns last 7 days of dietary logs.

Field Reference

Field	Type	Rule
`id`	int	auto-increment PK
`created_at`	ISO 8601 string	Always include `+05:30` timezone offset, never naive
`day_number`	int ⩾ 1	`(scan_date − program_start_date).days + 1`
`acne_score` / `redness_score` / `dryness_score`	float 0.0–10.0	**Lower = healthier** everywhere in the app
`ita_value`	float −30 to +55	Internal fairness context only — never display as a user-facing score
`local_aqi`	int 0–500+	India National AQI (CPCB) scale, not US EPA — convert server-side
`aqi_band`	enum	`good` · `satisfactory` · `moderate` · `poor` · `very_poor` · `severe`
`dominant_pollutant`	enum	`pm25` · `pm10` · `o3` · `no2` · `so2` · `co`
`evidence_tier`	enum	`clinical` · `mixed` · `traditional` — always visible in UI
`trend`	enum	`improving` · `flat` · `worsening` · `insufficient_data`
`nutrient_scores`	dict	Full continuous relevance map — all 10 nutrients scored every scan
`foods_to_add` / `foods_to_reduce`	array of `{name, reason, gi?, nutrient_value?, unit?}`	Personalized — filtered against user's recent dietary log

Critical: no `image` field in any response. Images are discarded on the server immediately after CV processing — they never round-trip to the client.

5. Recommendation Engine — How It Works

This is the biggest architectural decision in the project. Here is the complete pipeline, step by step.

Why not fixed rules?

A fixed rule set ("acne > 6 → show glycemic rule") has two fatal problems:

1. Everyone with the same score gets the same food list, regardless of what they're already eating.
2. There's a hard ceiling — you can only cover combinations you manually wrote.

Option B — Published coefficient scoring (implemented)

Instead of rules that "fire," every nutrient gets a continuous relevance score computed from the skin scores using published dermatology coefficients.

```
acne_score × coefficient_for_this_nutrient
+ redness_score × coefficient_for_this_nutrient
+ dryness_score × coefficient_for_this_nutrient
× AQI modifier (for antioxidant nutrients only)
= relevance score for this nutrient
```

Published sources for coefficients:

Nutrient target	Primary coefficient source
`low_glycemic` (acne weight: 0.68)	Fabbrocini et al. 2020, AJCN — strongest diet-acne predictor
`dairy_reduce` (acne weight: 0.41)	Fabbrocini 2020 — moderate, mixed evidence
`zinc` (acne: 0.35, redness: 0.28)	Jung et al. 2014, Dermatology
`omega3` (redness weight: 0.52)	Vaughn et al. 2018 systematic review
`antioxidant_vit_c` (redness: 0.30 × AQI mod)	Krutmann 2017 + 2024 PM2.5/acne meta-analysis
`antioxidant_vit_e` (dryness: 0.35 × AQI mod)	Krutmann et al. 2017, J Dermatol Sci
`healthy_fats` (dryness weight: 0.61)	General dermatology nutrition consensus
`probiotic` (acne weight: 0.30)	Bowe & Logan 2011, gut-skin axis
`vitamin_d` (all metrics: 0.20–0.25)	Nair & Maseeh 2012
`water_intake` (dryness weight: 0.55)	General dermatology consensus

AQI modifier (applied to antioxidant targets only):

- AQI good (0–50): × 1.0
- AQI satisfactory (51–100): × 1.1
- AQI moderate (101–200): × 1.2
- AQI poor (201–300): × 1.4
- AQI very poor (301–400): × 1.6
- AQI severe (400+): × 1.8

Result: a ranked list of all 10 nutrients with individual scores. No ceiling — any combination of scores produces a unique ranked list.

Personalization via dietary log filtering

Top 3 nutrient targets are passed to `nutrient_lookup.py`, which:

1. Returns the master food list for each nutrient (sourced from INDB for micronutrient rules, GI literature for glycemic rules)
2. Filters out any food the user logged eating in the last 3 days
3. Returns only foods they don't already eat regularly

Example (tested and confirmed working): Rohan logs: white rice, dal, curd, poha. System fires `low_glycemic` target.

- White rice: filtered out (already in his log)
- Dal: filtered out
- Curd: filtered out
- Poha: filtered out AND added to reduce list (GI ~70, commonly thought healthy)
- Result: recommends rajma, horse gram, chana, jowar roti — things he doesn't eat

Trend layer

Three recommendation layers total:

1. **State layer** — today's nutrient scores (which target ranks highest)
2. **Trend layer** — rolling 7-scan window comparison, smoothed:
   - `improving` — current score > 0.5 below rolling average
   - `worsening` — current score > 0.5 above rolling average
   - `flat` — within ±0.5 of rolling average
   - `insufficient_data` — fewer than 3 past scans (first ~7 days)
3. **Environmental layer** — AQI modifier already baked into nutrient scores; redness + AQI correlation surfaces antioxidant targets higher automatically

Escalation logic (Phase 2 extension — not yet built)

If a nutrient target has been top-ranked for 7+ days and trend is `flat`, the system should diversify to the next-ranked target rather than showing the same recommendation again. Plan for this before Phase 4 ships.

---

6. CV Pipeline — What Gets Measured & How

Metric	Method	Why this method	Dataset
Acne score (0-10)	Deep learning — MobileNetV2 or YOLOv8-Nano fine-tuned on ACNE04	Only metric with a properly labeled severity dataset; lesion texture/size complexity requires learned features	ACNE04 (ICCV 2019 benchmark)
Redness score (0-10)	Classical CV — LAB/HSV color-space deviation in OpenCV	Redness is a color signal, not shape/texture; no labeled dataset exists or is needed	No dataset — computed directly
Dryness score (0-10)	Classical CV — texture roughness + specular highlight analysis in OpenCV	True hydration requires a corneometer; cameras can only proxy visible dryness/flaking	No dataset — computed directly
ITA value (fairness)	Individual Typology Angle from L*a*b* pixel values	Standard dermatology-CV technique; avoids Fitzpatrick17k's known label-quality issues	No dataset — computed from pixel math

Acne model training notes:

- ACNE04 has imbalanced severity distribution (mild cases dominate) — use class-weighting or augmentation during training, not raw accuracy as metric
- Validate generalization on a labeled subset of DermaCon-IN to confirm the model works on Indian/Fitzpatrick III-VI skin, not just the training distribution
- **Phase 3 fallback:** if training time runs short, ship redness + dryness (classical CV, already implementable) and use a simple OpenCV lesion-count heuristic for acne. This is demoable and honest; a half-trained model is worse.

Where mock_cv.py fits: The entire CV pipeline sits behind one function in `app/mock_cv.py`. During Phases 1–2, it returns random plausible values. In Phase 3, you replace the function body with the real model. Nothing else in the backend changes — the API contract, recommender, and database schema are all stable.

---

7. 30-Day Adaptive Program

Program structure — elapsed time, not scan count:

- `program_start_date` set on first scan
- Progress shown as "Day X of 30" regardless of how many scans were done
- 2–3 day scanning cadence expected; daily not required
- Scan gaps are not missing data — last known scores carry forward for trend

Milestone check-ins (Flutter screens should handle these states):

Milestone	What to show
Day 1	Baseline scores; first recommendations; "your 30-day journey starts now"
Day 7	Which metric moved most; trend classification per metric
Day 14	Mid-program summary; escalate if any metric is still flat
Day 30	Full before/after comparison (first scan vs. latest); offer to renew

Smoothing rule (critical for avoiding noise-driven whiplash): Never compare a single scan to a single past scan. Always compare today's score against the rolling average of the last 3+ scans. A 3-point spike from bad lighting is noise; a sustained trend over 7+ days is signal.

Dermatologist escalation note: If redness or acne trend is `worsening` for 10+ consecutive days despite recommendations being followed, surface a note: "This pattern may benefit from a dermatologist's assessment — dietary changes alone may not be sufficient." This is the ethical boundary of what the app should claim.

---

8. Privacy & Image Lifecycle

Core principle: process and discard. Numbers persist; pixels don't.

Lifecycle (enforced in current backend)

```
1. Image uploaded via POST /api/scan/
2. Read into memory (await image.read())
3. Passed to CV pipeline (mock_cv.py or real model)
4. Score vector extracted
5. Image object goes out of scope → garbage collected
6. SkinScan record saved with scores only — no image field exists
7. API response returns scores — no image field in response
```

There is no image column in the `skin_scans` table. This is not an omission — it is the design. Images cannot be retained because there is nowhere to store them.

User-facing controls (Flutter, Phase 4)

- Consent screen before first camera use: *"Your photo is analyzed and immediately deleted. We only keep your skin scores, not the image."*
- Settings toggle: "Keep my scan photos" — **off by default**, opt-in only
- "Not a medical diagnosis" text visible wherever scores are displayed

Why this also saves money

Image: 1–5MB per scan. Score record: < 1KB per scan. Over 30 days: ~75–150MB of images vs. ~30KB of scores. This keeps the app on free-tier hosting for far longer.

Legal note (not legal advice)

India's Digital Personal Data Protection Act (DPDPA) includes data-minimization principles. "Process and discard" is the correct default posture for biometric-adjacent data regardless of whether strict compliance is required for a class project.

---

9. Pollution-Adjusted Skin Insights (New USP)

This is the project's strongest evidence-based differentiator.

Why it's clinically defensible

- PM2.5 exposure is significantly associated with increased skin redness area — confirmed by 2025 imaging-based assessment study
- Elevated PM2.5, PM10, and NO2 are associated with increased acne outpatient visits; pollutants are recognized in the "acne exposome" (2019 dermatology review board)
- 2025 systematic review confirms consistent pollution-acne associations across PM2.5, PM10, NO2, and traffic emissions
- PM2.5 triggers oxidative stress → depletes antioxidants → vitamin C/E recommendations are mechanistically justified, not just correlated

This means the pollution USP is in the `clinical` evidence tier — stronger than the dairy-acne link (which is `mixed`).

Technical implementation

- **Data:** WAQI API — free token, covers 1000+ Indian cities
- **City-level only:** never request precise GPS. AQI is a city/station metric anyway; fine-grained location adds no accuracy and adds privacy risk
- **AQI modifier** in `recommender.py`: automatically amplifies antioxidant nutrient targets (vit C, vit E) by 1.2–1.8× when AQI is above `moderate`
- **Smoothing:** rolling 3–7 day AQI average, same as scan trend — one bad air day shouldn't whiplash the recommendations

What the user sees

When AQI is `poor` or worse for multiple days:

- AQI badge on dashboard turns red with a short note
- `antioxidant_vit_c` and `antioxidant_vit_e` targets score higher automatically → amla, guava, drumstick leaves, almonds surface
- Card text explicitly links the AQI to the recommendation: *"Bhopal's air quality has been poor this week. PM2.5 at this level depletes your skin's antioxidant reserves —*

these foods help replenish them."

---

10. Flutter App — Screen Map

Screen	Purpose	API calls	Key components
Consent / Onboarding	First-launch privacy explanation + program start	None (local only)	Consent text, "Start my 30 days" button, disclaimer
Home / Dashboard	Today's scores, AQI badge, top recommendations, "New Scan"	`GET /api/history/` (latest scan)	Gauge widgets (acne/redness/dryness), AQI badge, recommendation cards
Capture	Camera with face alignment overlay	None (pre-upload)	`camera` package, alignment guide, capture button
Result	Scores just returned + full recommendation cards	`POST /api/scan/`	Score gauges, 3 NutrientRecommendation cards each showing evidence tier + foods
Trend / History	7-day line chart + Day X of 30 progress	`GET /api/history/`	`fl_chart` line chart, 3 lines (acne/redness/dryness), milestone markers at Day 7/14/30
Diet Log	User logs what they ate	`POST /api/log/`, `GET /api/log/`	Text input or checkbox from common foods list
Recommendation Detail	Full explanation + source note for one recommendation	(from Result screen state)	Body text, evidence tier badge, source note, full foods list

Evidence tier color coding (consistent across all screens):

- `clinical` → green badge
- `mixed` → amber badge
- `traditional` → terracotta/rust badge

"Not a medical diagnosis" disclaimer: visible on Result screen, Home screen near scores, and in the app footer — not a one-time popup.

---

11. Development Roadmap

Phase 1 — Schema & Contract (Week 1–2)

Do together before splitting:

- Lock the API contract (§4) — write it down, both agree, don't deviate
- Decide persistence fix (Render disk vs Postgres) — do this now not at deploy
- Register WAQI token at aqicn.org/api
- Person A: scaffold FastAPI project, `python seed_rules.py` to confirm server boots
- Person B: `flutter create twacha`, add Dio + Provider + fl_chart, scaffold all 7 screens with dummy in-app data

Phase 1 is done when: both of you can hit `POST /api/scan/` from the Flutter app and see fake-but-correctly-shaped scores render on the Result screen.

Phase 2 — Recommender & Dietary Log (Week 3–5, parallel with Phase 3)

Person A (backend):

- Expand nutrient evidence rules to 10–15 (currently 10 nutrients in the scorer)
- Build `POST /api/log/` and `GET /api/log/` (already built in v2)
- Expand `nutrient_lookup.py` food lists — review each entry against INDB
- Add escalation logic: if top target has been flat 7+ days, surface next target

Person B (Flutter):

- Wire Diet Log screen to `POST /api/log/`
- Display evidence tier badges on recommendation cards
- Implement AQI badge color coding on dashboard

Phase 3 — Vision Model (Week 3–8, parallel with Phase 2)

Person A (ML track, Colab):

1. Download ACNE04 from Roboflow or Hugging Face
2. Inspect 50 images — check label distribution, decide classification vs detection
3. Fine-tune MobileNetV2/EfficientNet (classification) on ACNE04 severity grades
4. Separately implement OpenCV redness (LAB color-space) + dryness (texture) — ship these first, they take days not weeks and give an early demoable win
5. Compute ITA from L*a*b* pixel values
6. Validate on DermaCon-IN subset for Indian-skin generalization
7. Replace `mock_analyze_image()` in `app/mock_cv.py` — nothing else changes

Fallback plan (if training doesn't converge in time): Ship redness + dryness (OpenCV, days of work) + OpenCV lesion-count heuristic for acne. A working, honest system beats an ambitious broken one.

Phase 4 — Integration & Polish (Week 7–10)

- Swap mock CV scores for real model output (only `mock_cv.py` changes)
- Consent screen + "not a diagnosis" text in place before any real photo
- Day 7/14/30 milestone screens
- 5–10 test users, collect feedback
- Evaluate: precision/recall on ACNE04 held-out test split; user survey on recommendation relevance; acne score trend over full 30 days for any user who completes the program

---

12. Backend — What Is Built & Working

Version 2 (current) — tested end-to-end, not just written:

File	Status	What it does
`app/database.py`	✅ Working	SQLAlchemy engine + session, switches to Postgres via `DATABASE_URL` env var
`app/models.py`	✅ Working	`SkinScan`, `Program`, `DietaryLog`, `NutrientEvidenceRule`
`app/schemas.py`	✅ Working	Pydantic models enforcing the §4 contract — includes `nutrient_scores` map
`app/recommender.py`	✅ Working	10-nutrient continuous scoring engine with published coefficients + AQI modifier + trend calc
`app/nutrient_lookup.py`	✅ Working	Master food database (INDB + GI literature sourced) with dietary-log filtering
`app/aqi_client.py`	✅ Working	WAQI integration with automatic mock fallback when no token set
`app/mock_cv.py`	✅ Working (stub)	Returns random plausible scores — swap body for real model
`app/routers/scans.py`	✅ Working	Full pipeline: CV → AQI → score nutrients → filter foods → trend → save → respond
`POST /api/scan/`	✅ Tested	Confirmed returning correct JSON including filtered food lists
`GET /api/history/`	✅ Tested	Returns program + scan list
`POST /api/log/`	✅ Tested	Upserts daily food log
`GET /api/log/`	✅ Tested	Returns recent logs
Trend calculation	✅ Tested	Confirmed `improving` fires correctly against synthetic history
Dietary filtering	✅ Tested	Confirmed white rice/dal/curd excluded from recommendations when logged

What is genuinely mocked (not yet real):

- `mock_cv.py` — returns random scores, not trained model output
- WAQI fallback fires because no token is set in the sandbox — real token needed

Environment variables to set before running:

```
WAQI_TOKEN=your_token_from_aqicn.org
DATABASE_URL=sqlite:///./twacha.db   (default, or swap for Postgres URL)
```

To run:

bash

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python -m uvicorn app.main:app --reload
# visit http://127.0.0.1:8000/docs for Swagger UI
```

---

13. Datasets & Data Sources

Source	Type	Use	License / Access
ACNE04	Download	Primary CV training — acne severity grades + lesion bounding boxes	Research use; available on Roboflow, Hugging Face, GitHub (Acne04-v2 for cleaned labels)
DermaCon-IN	Download	Indian-skin validation set — 5,450 clinical images, South Indian	CC BY-NC-SA 4.0 — non-commercial only
INDB (Indian Nutrient Databank)	Download	Nutrient composition of 1,095 Indian foods + 1,014 recipes	Open-access for research — safer than raw IFCT for building on
International GI Tables (Atkinson et al. 2008, AJCN)	Literature	GI values for Indian staples (jowar, bajra, rajma, chana etc.) — NOT in INDB	Access via PubMed; India-specific GI study CSV extracts on GitHub
WAQI API (aqicn.org)	Live API	Real-time AQI, PM2.5, PM10, NO2 for Indian cities	Free token registration at aqicn.org/api
PubMed systematic reviews	Literature	Evidence base for recommendation coefficients — manual curation	Free access; 15–20 papers cover all 10 nutrient targets
Redness + dryness	None needed	Classical OpenCV — computed from pixel values directly	—
ITA value	None needed	Computed from L*a*b* pixel math directly	—

Critical distinction: INDB ≠ GI literature INDB gives you nutrient composition (zinc mg/100g, vitamin C mg/100g etc.). It does not give you glycemic index values — those come from separate clinical studies measuring blood glucose response. Tag each food entry in `nutrient_lookup.py` with its source so the evidence trail is clear.

Dataset to deprioritize:

- SCIN (Google): US-based, ~14% Fitzpatrick V/VI — doesn't support your India-fairness claims despite appearances
- Fitzpatrick17k: known label-quality issues from atlas scraping — use ITA computed directly from pixels instead

14. Evaluation Plan

CV model (Phase 3):

- Primary metric: intraclass correlation coefficient (ICC) between model severity scores and dermatologist-graded severity on a held-out ACNE04 test split
- Fairness check: compare model accuracy across ITA ranges on the DermaCon-IN validation subset — flag if lighter-skin ITA bands score significantly better

Recommender (Phase 2):

- Manual review: domain-literate reviewer (nutrition/derm student is sufficient) checks each of the 10 nutrient targets and their food lists before scaling
- Logic test: force fixed score vectors (e.g., acne=9, redness=2, dryness=2) and confirm glycemic target ranks first; (acne=2, redness=9, dryness=2) confirms omega3/antioxidant rank first

End-to-end (Phase 4):

- 5–10 test users doing the full 30-day program
- Collect: scan scores over time, dietary logs, subjective skin rating
- Report: did scores trend downward for users who followed recommendations? Is there a measurable difference between users who logged diet vs. those who didn't (dietary filtering active vs. inactive)?

---

15. Do / Don't

Do:

- Lock the API contract in Week 1 before either person writes a single screen
- Build Flutter screens against mock backend data immediately — don't wait for Phase 3
- Ship redness + dryness (classical CV) before the acne model is trained — early demo milestone
- Use city-level location only for AQI — never precise GPS
- Smooth all trend calculations over multiple scans, not single-point comparisons
- Keep "not a medical diagnosis" disclaimer visible wherever scores appear
- Default image retention to off — make it opt-in if users want it
- Delete the uploaded image in the same request cycle as the CV processing
- Tag every food recommendation with its data source (INDB vs GI literature)
- Get a domain-literate reviewer to check nutrient-food mappings before scaling

Don't:

- Use DermaCon-IN diagnosis labels as severity scores — different task
- Treat SCIN as your Fitzpatrick III-VI solution — it skews to lighter tones

- Mix `clinical`, `mixed`, and `traditional` evidence tiers without labeling them
- Scope-creep the nutrient target list past 10–12 — you already have all the key signals
- Let a single AQI spike change recommendations — use rolling average
- Let a single scan score whiplash the trend — smooth over 3+ scans
- Skip Alembic migrations — set up `alembic init` before schema stabilizes
- Decide "Render disk vs Postgres" at Phase 4 — decide it now
- Claim the app measures true skin hydration from a photo — it measures visible dryness

---

Blueprint version: Final · Last updated with v2 backend architecture. Next steps: Phase 3 CV model training (ACNE04) + Flutter screen wiring (Phase 4).